AULA COMPLETA

Algoritmos de Ordenação e Filas de Prioridade

Slides reunidos a partir das capturas de tela da aula

25 capturas organizadas • dos conceitos ao comparativo final

Imagens preservadas integralmente • sem cortes

Roteiro da aula

3 horas

Nosso roteiro de hoje

01
Boas-vindas
Instrutores e turma

02
O problema
Bilheteria + conceitos: estabilidade, memória, Big-O

03
Insertion & Selection
Os métodos “do dia a dia”

04
Merge & Quick
Dividir para conquistar

05
☕ Intervalo
20 a 30 minutos

06
Heap & Heap Sort
A árvore que sabe quem é o maior

07
Filas
Fila, deque e fila de prioridade

08
Bilheteria resolvida
+ comparativo: qual escolher?

09
Quiz
5 perguntas rápidas

10
Resumo + próximas aulas
O que levamos e o que vem nos Dias 2 e 3

Ao final de hoje, você vai conseguir...

Explicar com suas palavras como 5 algoritmos de ordenação funcionam, comparar quando usar cada um, e entender como uma fila de prioridade decide “quem é o próximo”.

Combinados

Pode perguntar a qualquer hora pelo chat ou microfone. Não existe pergunta boba: ninguém aqui precisa saber programar.

Problema gerador

A bilheteria do show

50.000 pedidos disputam **20.000 ingressos**. Os pedidos chegam de três tipos de cliente:

- Agências · prioridade 1
- Assinantes · prioridade 2
- Público geral · prioridade 3

As regras

1. Primeiro as classes mais altas.
2. Dentro da mesma classe, **quem chegou antes** compra antes.
3. Um cliente pode virar assinante no meio da fila (**upgrade**).
4. Alguém pode **cancelar** o pedido a qualquer momento.

A grande pergunta

Como o sistema decide, rápido e sem errar, **quem é o próximo** a comprar?

Pense comigo

- Dá para colocar tudo em ordem uma vez só e pronto?
- E se chegar um pedido novo, reordenamos os 50.000?
- E se alguém fizer upgrade?

Guarde suas respostas: vamos voltar a este problema no fim da aula.

Chrome File Edit View History Bookmarks Profiles Tab Window Help Wed 14 Oct 20:06

Ordenação & Filas de Prioridade | Coding Tank | Aula online — C

/Users/guilhermesoares/ada/aula.html#s6

Ask Gemini

Conceito

Estabilidade: o que acontece com os empates?

Um algoritmo é **estável** quando itens com o mesmo valor **mantêm a ordem em que estavam**.



1. Ordem de chegada | **2. Ordenar por idade (estável)** | **3. Ordenar por idade (instável)**

Ordem de chegada. Repare: **Bia** e **Ana** têm a mesma idade (30), e Bia chegou antes.

Na bilheteria isso importa: dois assinantes empatados na classe devem continuar na **ordem de chegada**.

Conceito

In-place x out-of-place: quanto espaço extra?

📚 In-place ("no próprio lugar")



Arrumamos os livros **na mesma estante**, trocando de lugar. Só precisamos de uma mão livre.

Memória extra: `O(1)`, uma quantidade fixa. Ex.: Insertion, Selection, Quick, Heap.

🗄️ Out-of-place ("em outro lugar")



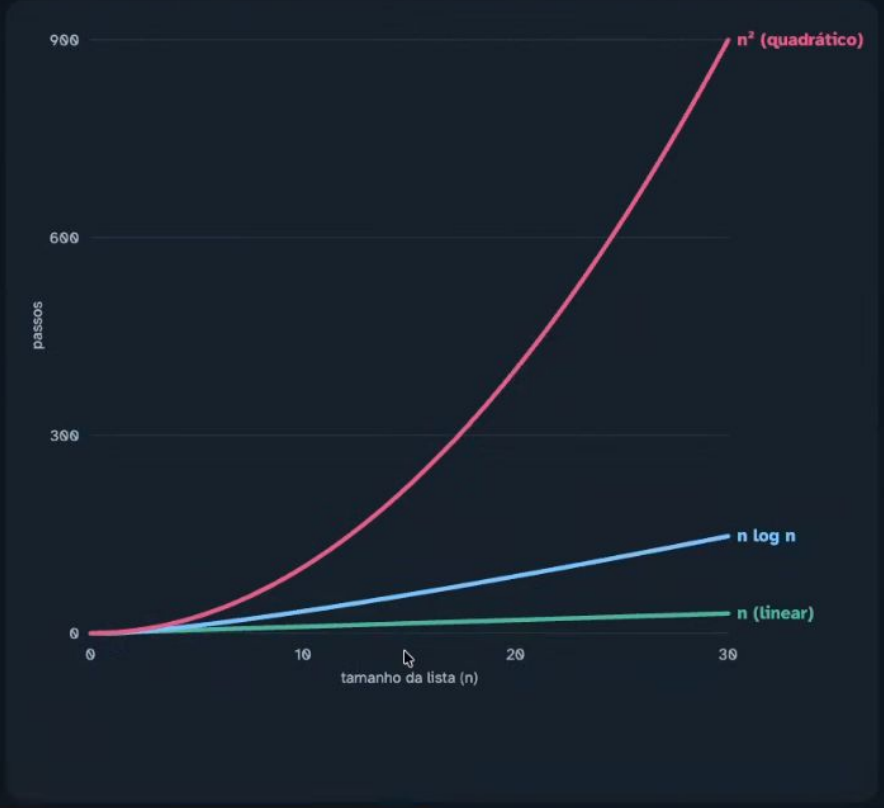
Usamos **uma segunda estante** para ir montando a ordem, e depois devolvemos.

Memória extra: `O(n)`, cresce junto com a lista. Ex.: Merge Sort.

Trade-off: às vezes gastar **mais memória** compra **mais velocidade** ou **estabilidade**.

Conceito

Big-O: quanto o trabalho cresce?



Big-O não mede segundos. Ele responde: **se a lista dobrar de tamanho, o que acontece com o trabalho?**

Na bilheteria (n = 50.000)

O(n²)	2,5 bilhões	≈ 42 min*
O(n log n)	≈ 780 mil	≈ 0,8 s*
O(log n)	≈ 16	instantâneo

* supondo 1 microssegundo por passo.

O(1) constante · **O(log n)** "cortar pela metade" · **O(n)** olhar cada um · **O(n²)** comparar todos com todos.

Os cinco algoritmos da aula

Visão geral

As 5 receitas de hoje

1	2	3	4	5
Insertion Sort	**Selection Sort**	**Merge Sort**	**Quick Sort**	**Heap Sort**
Ordenar "encaixando"	Escolher o menor, sempre	Dividir e juntar	Escolher um pivô e separar	O maior sobe ao topo

Como cada tela funciona

À esquerda, a explicação: 💬 **em palavras simples** ou ⚙ **visão técnica** (pseudocódigo, Big-O, estabilidade). À direita, **3 exemplos animados** para seguir passo a passo.

Como ler as cores

● em mãos / menor ▲ comparando ◆ pivô ✔ em ordem

Cada cor vem com um símbolo e um texto: ninguém depende só da cor.

Algoritmo 1 de 5

Insertion Sort — ordenar "encaixando"

Em palavras simples | Visão técnica

Cartas na mão | Fila por idade | Coleção de livros

A ideia em uma frase

Pegamos um item de cada vez e o **encaixamos no lugar certo** entre os que já estão arrumados.

Como funciona

1. A parte da esquerda já está em ordem.
2. Pegamos o próximo item.
3. Andamos para a esquerda até achar onde ele cabe.

No seu dia a dia

É o que você faz ao **organizar cartas de baralho na mão**: recebe uma carta nova e a encaixa entre as que já estão em ordem.

Bom quando: a lista é pequena ou já está quase em ordem.

Você recebeu 5 cartas e quer deixá-las em ordem crescente na mão.



O primeiro item (7 de copas) sozinho já está "em ordem".

Comparações: 0 · Deslocamentos: 0

◄ Voltar | Próximo passo ► | ► Automático | ↻ Recomeçar

Passo 1 de 24

Fila organizada por idade



Insertion Sort na bilheteria

Teste na bilheteria

E se usarmos o Insertion Sort na bilheteria?

COMPARAÇÕES

≈ 625 milhões

para ordenar 50.000 pedidos

TEMPO ESTIMADO

≈ 10 min

caso médio

MEMÓRIA EXTRA

Nenhuma

O(1): só a "chave"

RESPEITA A ORDEM DE CHEGADA?

✔ Sim

quem chegou antes continua antes

PESO PARA A BILHETERIA — **Pesado**

Pesado para ordenar os 50.000 de uma vez. Mas é ótimo para **encaixar pedidos que chegam um a um**.

Placar de peso

escala que cresce de 10 em 10 vezes



▶ Ordenar os 50.000 pedidos

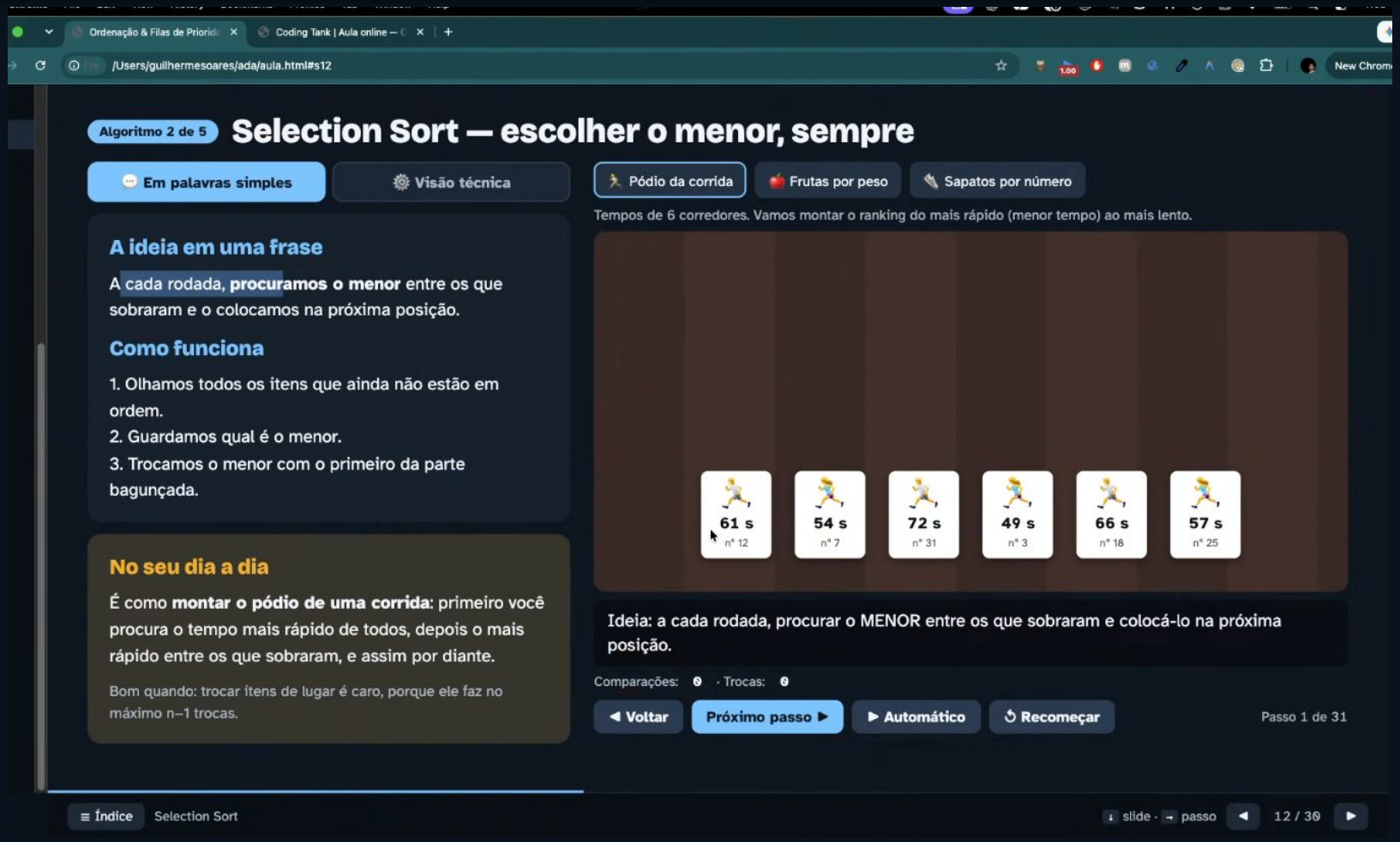
0 comparações · 0 s

Simulação acelerada: 1 segundo aqui = 1 minuto real (1 comparação = 1 microssegundo).

E quando a fila muda?

📥 **Chega um pedido novo**	Encaixa direto no lugar: até 50 mil deslocamentos **(≈ 0,05 s)**
⬆️ **Alguém faz upgrade**	Tira e encaixa de novo **(≈ 0,05 s)**
✖ **Alguém cancela**	Tira e fecha o buraco **(≈ 0,05 s)**

Ordenando corredores por tempo



Teste na bilheteria

E se usarmos o Selection Sort na bilheteria?

COMPARAÇÕES
≈ 1,25 bilhão
para ordenar 50.000 pedidos

TEMPO ESTIMADO
≈ 21 min
em TODOS os casos

MEMÓRIA EXTRA
Nenhuma
O(1)

RESPEITA A ORDEM DE CHEGADA?
✗ Não
pode trocar a ordem de quem chegou antes

PESO PARA A BILHETERIA **Muito pesado**

O mais pesado: sempre faz o trabalho completo, mesmo com a lista quase em ordem. **Inviável** para a bilheteria.

Placar de peso
escala que cresce de 10 em 10 vezes



▶ Ordenar os 50.000 pedidos 0 comparações · 0 s

Simulação acelerada: 1 segundo aqui = 1 minuto real (1 comparação = 1 microssegundo).

E quando a fila muda?

📥 **Chega um pedido novo**	Não sabe encaixar: roda tudo de novo **(≈ 21 min)**
⬆️ **Alguém faz upgrade**	Roda tudo de novo **(≈ 21 min)**
✖ **Alguém cancela**	Tirar é fácil **(≈ 0,05 s)**

Algoritmo 3 de 5

Merge Sort — dividir e juntar

Em palavras simples | Visão técnica

Provas por nota | Preços no mercado | Playlist por duração

O professor quer as provas da menor para a maior nota, para chamar primeiro quem precisa de reforço.

A ideia em uma frase

Dividimos a lista ao meio até sobrarem pedaços de 1 item, e depois **juntamos** os pedaços sempre em ordem.

Como funciona

1. Quebramos a lista ao meio, várias vezes.
2. Um pedaço de 1 item já está ordenado.
3. Juntamos dois pedaços comparando sempre o primeiro de cada um.

No seu dia a dia

É como **juntar duas pilhas de provas já ordenadas**: você olha a prova do topo de cada pilha e sempre pega a de menor nota.

Bom quando: a lista é grande e você precisa de estabilidade e de tempo garantido.

LISTA

7,5	4,0	9,0	6,5	8,0	5,5
Ana	Bruno	Carla	Davi	Elis	Fábio

ÁREA AUXILIAR

Estratégia “dividir para conquistar”: quebrar a lista ao meio até sobrarem pedaços de 1 item, e depois juntar em ordem.

Comparações: 0 · Movimentos para o auxiliar: 0

◄ Voltar | Próximo passo ► | ► Automático | ↺ Recomeçar

Passo 1 de 42

Teste na bilheteria

E se usarmos o Merge Sort na bilheteria?

COMPARAÇÕES
≈ 780 mil
para ordenar 50.000 pedidos

TEMPO ESTIMADO
≈ 0,8 s
em todos os casos

MEMÓRIA EXTRA
+50 mil
espaços na área auxiliar

RESPEITA A ORDEM DE CHEGADA?
✔ Sim
empates mantêm a ordem de chegada

PESO PARA A BILHETERIA Leve

Rápido e respeita a ordem de chegada. O peso aparece quando **a fila muda o tempo todo**.

Placar de peso · escala que cresce de 10 em 10 vezes



↻ Rodar de novo **780.000 comparações · 0,8 s**

Simulação acelerada: 1 segundo aqui = 1 minuto real (1 comparação = 1 microssegundo).

E quando a fila muda?

📥 **Chega um pedido novo**	Reordenar tudo **(≈ 0,8 s)**. Parece pouco, mas 50.000 pedidos × 0,8 s ≈ **11 horas**!
⬆️ **Alguém faz upgrade**	Reordenar tudo **(≈ 0,8 s)**
✖ **Alguém cancela**	Tira e fecha o buraco **(≈ 0,05 s)**

Escolher um pivô

Algoritmo 4 de 5

Quick Sort — escolher um pivô e separar

Em palavras simples | Visão técnica

Turma por altura | Entregas por distância | Cartas na mesa

A ideia em uma frase

Escolhemos um item como **pivô**, colocamos os menores à esquerda e os maiores à direita. Depois repetimos em cada lado.

Como funciona

1. Escolhemos um pivô (aqui, o último item).
2. Quem for menor que o pivô vai para a esquerda do "muro".
3. O pivô vai para o muro: posição final. Repetimos em cada lado.

No seu dia a dia

É como **organizar uma turma para a foto**: você escolhe uma pessoa de referência e diz "quem é mais baixo, para a esquerda; mais alto, para a direita".

Bom quando: uso geral. Costuma ser o mais rápido na prática.

Foto da turma: organizar do mais baixo para o mais alto.



Ideia: escolher um "pivô" e separar: menores para a esquerda, maiores para a direita. Depois repetir em cada lado.

Comparações: 0 · Trocas: 0

◄ Voltar | Próximo passo ► | ► Automático | ↻ Recomeçar

Passo 1 de 35

Teste na bilheteria

E se usarmos o Quick Sort na bilheteria?

COMPARAÇÕES
≈ 1,08 milhões
para ordenar 50.000 pedidos

TEMPO ESTIMADO
≈ 1 s
médio · pior caso ≈ 21 min!

MEMÓRIA EXTRA
≈ 16
"lembretes" de pedaços (O(log n))

RESPEITA A ORDEM DE CHEGADA?
✗ Não
precisa pôr a ordem de chegada na chave

PESO PARA A BILHETERIA Médio

Normalmente rápido, mas **arriscado**: os pedidos chegam quase em ordem, e isso pode levar ao pior caso.

Placar de peso · escala que cresce de 10 em 10 vezes



Rodar de novo **1.080.000 comparações · 1,1 s**

Simulação acelerada: 1 segundo aqui = 1 minuto real (1 comparação = 1 microssegundo).

E quando a fila muda?

Chega um pedido novo	Reordenar tudo (≈ 1 s), com risco de cair no pior caso
Alguém faz upgrade	Reordenar tudo (≈ 1 s)
✖ Alguém cancela	Tira e fecha o buraco (≈ 0,05 s)

Estrutura

Antes do Heap Sort: o que é um heap?

Um **heap** é uma árvore "organizada por andares" que obedece a duas regras:

1. Andares completos

Preenchemos andar por andar, da esquerda para a direita. Por isso ele cabe direitinho numa lista.

2. O mais importante fica em cima

Min-heap: todo pai é **menor** que os filhos (o menor está no topo).
Max-heap: todo pai é **maior** (o maior está no topo).

Como uma **empresa**: o chefe de cada equipe é sempre "mais prioritário" que sua equipe. Não importa a ordem entre colegas do mesmo nível.



Na lista: filhos de k ficam em **2k+1** e **2k+2**; o pai de k fica em **(k−1)/2**.

Ver o topo	`O(1)`	está sempre na posição 0
Inserir	`O(log n)`	entra no fim e "sobe"
Remover o topo	`O(log n)`	o último vai ao topo e "desce"

Heap Sort: o maior no topo

Algoritmo 5 de 5

Heap Sort — o maior sobe ao topo

Em palavras simples | Visão técnica

A ideia em uma frase

Organizamos os itens numa **árvore em que o maior fica sempre no topo**. Tiramos o topo, colocamos no fim, e repetimos.

Como funciona

1. Montamos um max-heap: todo pai é maior que seus filhos.
2. O topo é o maior: trocamos ele com o último e o tiramos da árvore.
3. Reorganizamos a árvore e repetimos.

No seu dia a dia

É como um **campeonato de chaves**: o campeão sobe ao topo. Depois que ele recebe o troféu, os outros disputam de novo quem sobe.

Bom quando: você precisa de tempo garantido sem memória extra.

Ranking de pontos | Pedidos por valor | Idades

Valores (em reais) de 7 pedidos de uma loja online, do mais barato ao mais caro.



JÁ ORDENADOS (SAÍRAM DA ÁRVORE)

lista: 120 [0] | 45 [1] | 300 [2] | 80 [3] | 210 [4] | 60 [5] | 150 [6]

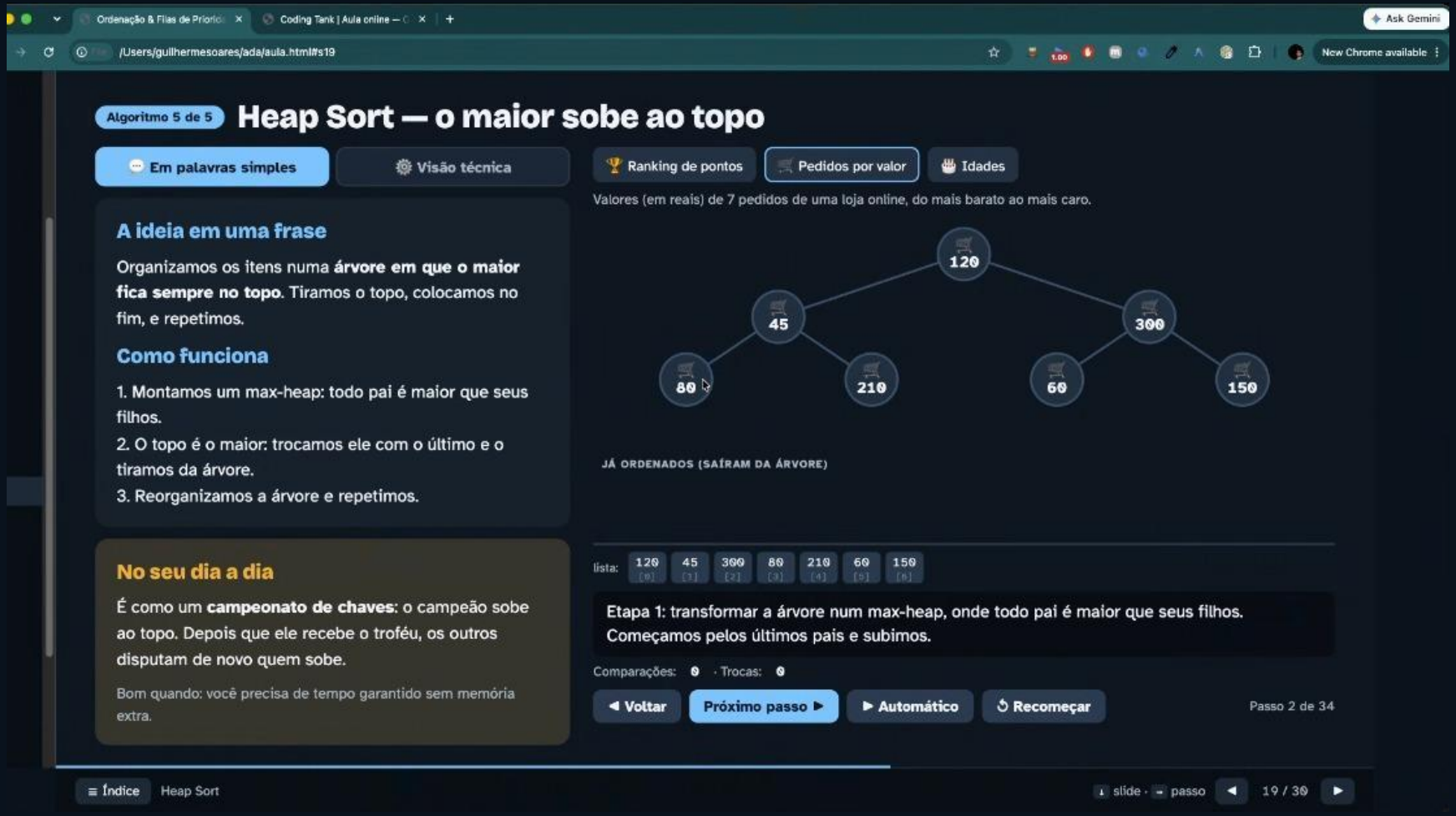
Primeiro, enxergamos a lista como uma árvore: a posição 0 é a raiz, e cada posição k tem os filhos 2k+1 e 2k+2.

Comparações: 0 · Trocas: 0

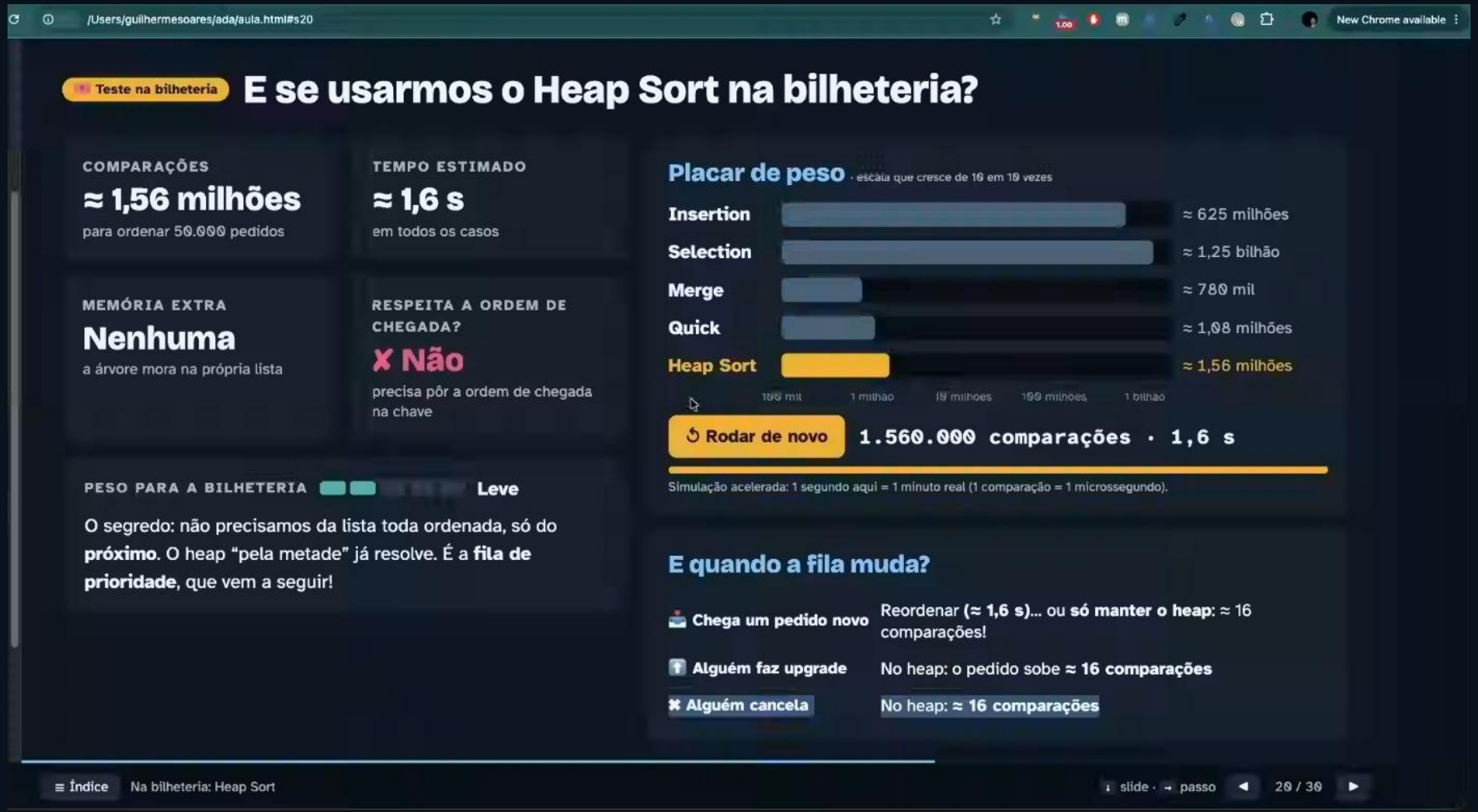
◄ Voltar | Próximo passo ► | ► Automático | ↺ Recomeçar

Passo 1 de 34

Construindo o max-heap



Heap Sort na bilheteria



Queue: primeiro a chegar

Filas

Fila comum (Queue): o primeiro a chegar é o primeiro a sair

Em palavras simples

É a fila do banco: quem chega vai para o **fim**; quem é atendido sai do **início**.

Visão técnica

Estrutura **FIFO** (*First In, First Out*).
enqueue: inserir no fim, O(1)
dequeue: remover do início, O(1)

Limitação: ela **não sabe de prioridade**. Uma emergência esperaria atrás de todo mundo.



+ Chegou alguém (enqueue) | – Atender o próximo (dequeue) | ↺ Recomeçar

Clique nos botões para ver a fila funcionando.

Ordenação & Filas de Prioridade | Coding Tank | Aula online — C

/Users/guilhermesoares/ada/aula.html#s21

Ask Gemini | New Chrome available

Filas

Fila comum (Queue): o primeiro a chegar é o primeiro a sair

Em palavras simples

É a fila do banco: quem chega vai para o **fim**; quem é atendido sai do **início**.

Visão técnica

Estrutura **FIFO** (*First In, First Out*).
enqueue: inserir no fim, O(1)
dequeue: remover do início, O(1)

Limitação: ela **não sabe de prioridade**. Uma emergência esperaria atrás de todo mundo.

CAIXA (INÍCIO) → FIM DA FILA



+ Chegou alguém (enqueue) | – Atender o próximo (dequeue) | ↺ Recomeçar

Bruno foi atendido(a): saiu pelo **início**. Primeiro a chegar, primeiro a sair. Fila: Carla → Davi → Elis → Fábio → Gabi → Hugo → Iara

Filas

Deque: uma fila com duas portas

Em palavras simples

Dá para **entrar e sair pelas duas pontas**. Ex.: no embarque, uma gestante entra pela frente e alguém que desiste sai pelo fim.

Visão técnica

Deque = *double-ended queue*. Inserir e remover no início ou no fim, tudo em O(1).

Uso real: o botão "voltar" do navegador e o "desfazer" (Ctrl+Z) de editores, com limite de histórico.



+ Entrar no fim | + Entrar no início | − Sair do início | − Sair do fim | ↺ Recomeçar

Lia entrou pelo **início** (fura-fila autorizado: ex. gestante). Fila: Lia → Hugo → Gabi → Carla → Davi

Filas

Fila de prioridade: o mais urgente primeiro

No pronto-socorro

Cada paciente recebe uma cor na triagem (Protocolo de Manchester). Quem é chamado é o **mais urgente**, não quem chegou primeiro.

1. Emergência
2. Muito urgente
3. Urgente
4. Pouco urgente
5. Não urgente

Visão técnica

Implementada com um **min-heap**: menor número = mais urgente. Empate? Ganha quem chegou antes.
Inserir: `O(log n)` · Atender: `O(log n)` · Ver o próximo: `O(1)`



lista: 1·Rafa [0] | 2·Ana [1] | 5·Davi [2] | 4·José [3] | 3·Lúci [4]

Clique em “Chegou paciente” ou “Atender o próximo”.

Chegou paciente | Atender o próximo | Recomeçar

Filas

Fila de prioridade: o mais urgente primeiro

No pronto-socorro

Cada paciente recebe uma cor na triagem (Protocolo de Manchester). Quem é chamado é o **mais urgente**, não quem chegou primeiro.

1. Emergência
2. Muito urgente
3. Urgente
4. Pouco urgente
5. Não urgente

Visão técnica

Implementada com um **min-heap**: menor número = mais urgente. Empate? Ganha quem chegou antes.
Inserir: `O(log n)` · Atender: `O(log n)` · Ver o próximo: `O(1)`



lista: 2·Ana [0] | 3·Lúci [1] | 5·Davi [2] | 4·José [3]

Está no lugar certo.

Chegou paciente | Atender o próximo | Recomeçar

Resumo

Comparativo: os trade-offs lado a lado

ALGORITMO	MELHOR	MÉDIO	PIOR	MEMÓRIA EXTRA	ESTÁVEL?	IN-PLACE?
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim	Sim
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não	Sim
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim	Não
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não	Sim
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não	Sim

Lembrete: não existe “o melhor algoritmo”. Existe o melhor para **cada situação**.